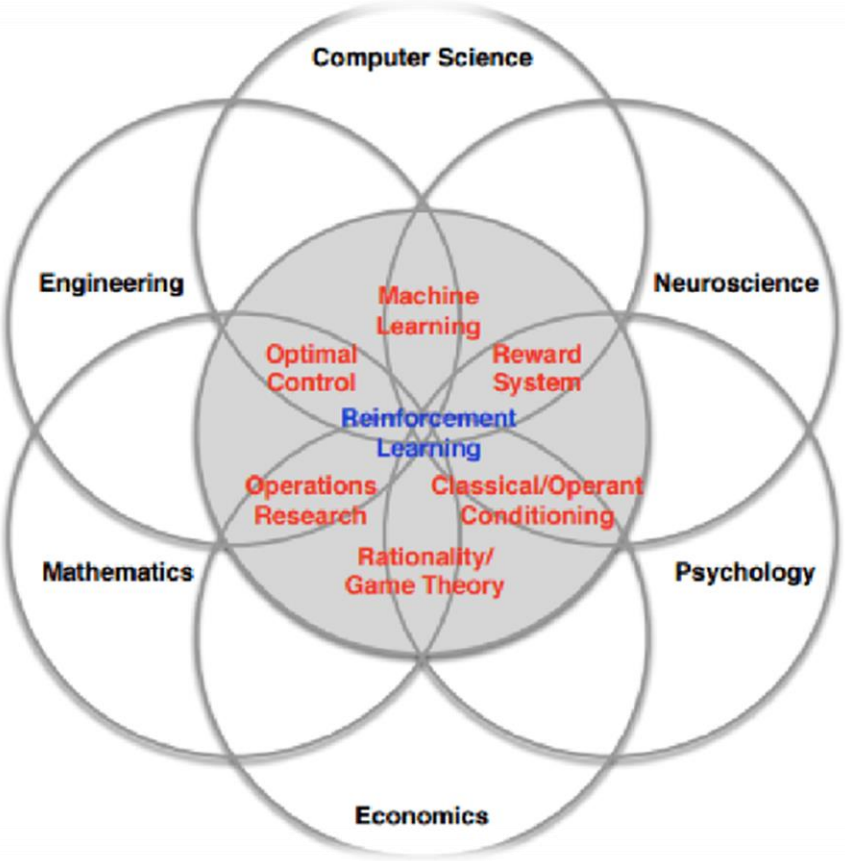
REINFORCEMENT LEARNING

PREMISE

- Learning how to map situations to actions, so as to maximize a numerical reward.
- Features:
  - Trial and error search
  - Delayed rewards
  - Dilemma of exploration vs exploitation
- Applications:
  - Games
  - Robotics

PREMISE

PARADIGM


Computer Science
Engineering
Neuroscience
Machine Learning
Optimal Control
Reward System
Reinforcement Learning
Operations Research
Classical/Operant Conditioning
Rationality/ Game Theory
Mathematics
Psychology
Economics

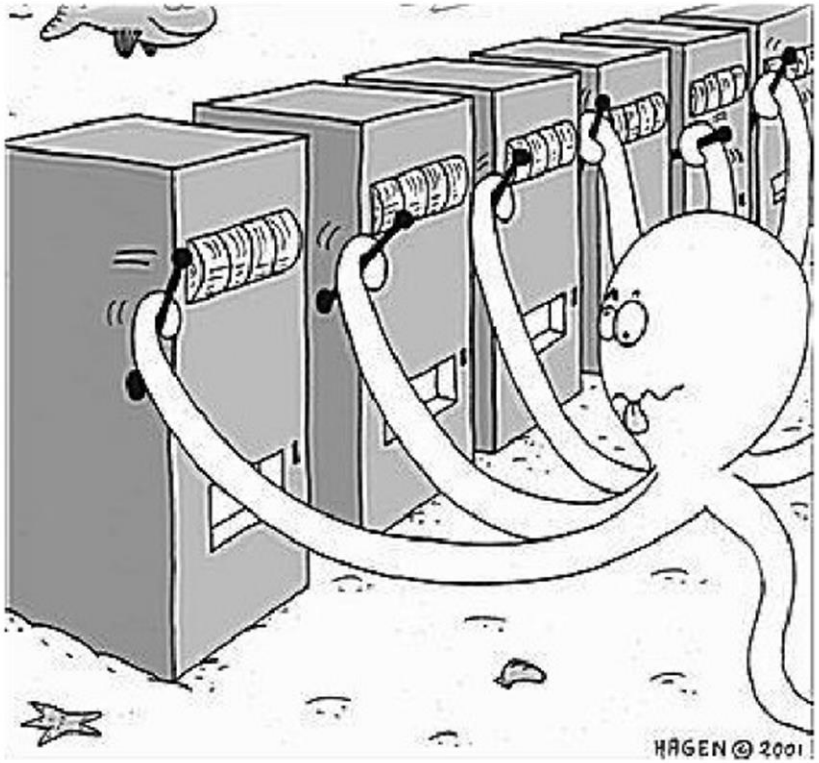
ELEMENTS OF R.L.

- Policy:
  - defines the agent's behaviour at a given time
  - it is a mapping from states to actions
  - can be stochastic
- Reward signal:
  - at each time step, the environment sends a number to the agent called the reward
  - ultimate goal of the agent is to maximize total reward

ELEMENTS OF R.L.

- Value function:
  - the value of a state is the total amount of reward the agent can expect to accumulate over the future, starting from that state
  - reward signal indicates what is good in the immediate sense; the value function indicates what is good in the long run
- Model (optional):
  - approximation of the environment that can be used to predict the next state and reward given the current state and action
  - used for planning
  - model-free methods vs model-based methods

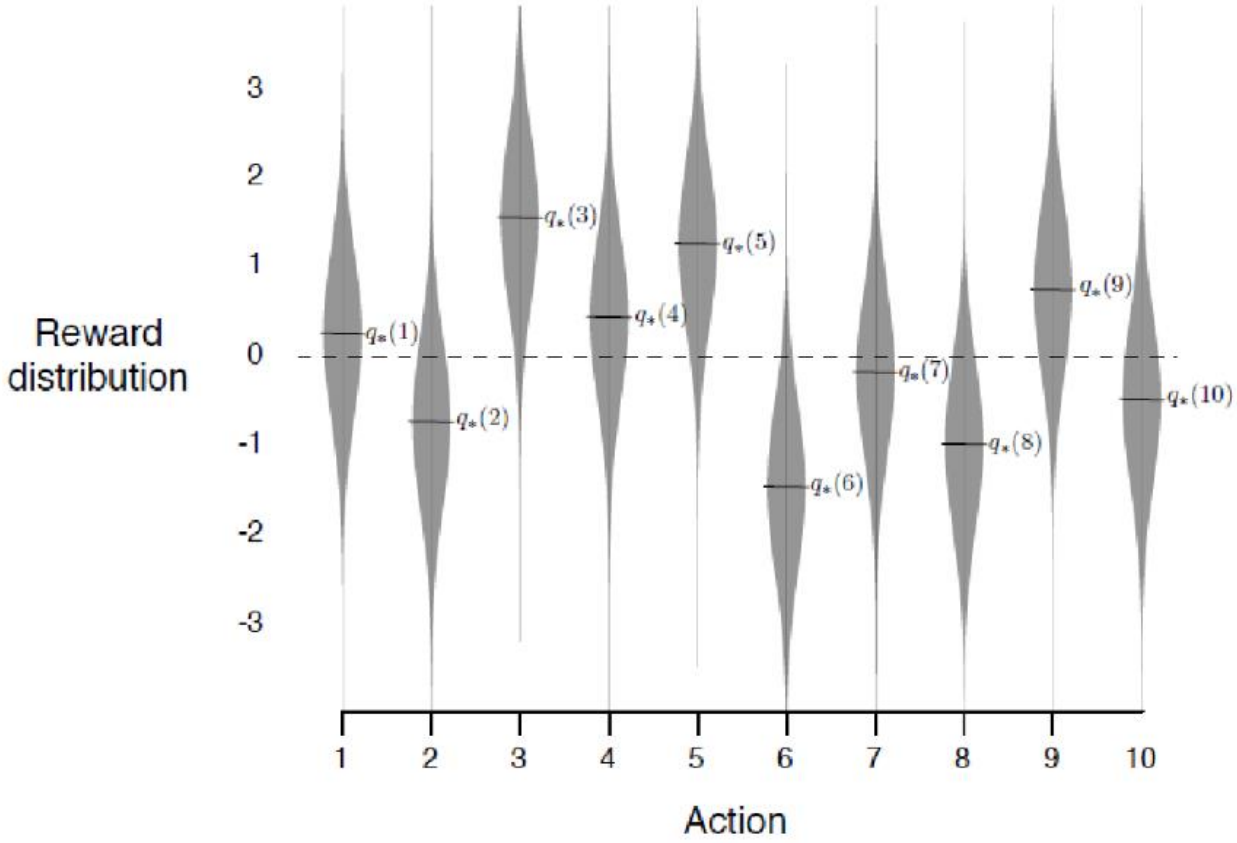
MULTI-ARMED BANDIT



MULTI-ARMED BANDIT

- Imagine there are k jackpot machines in front of you. When you pull the lever of machine i, with some unknown probability p_i you will win \$1, and with probability $1 - p_i$ you will receive nothing.
- The average payouts of the machines may all be different, and the casino affords you 1000 lever pulls before asking you to leave. What should your strategy be?
- Here we are assuming the rewards are distributed as Bernoulli random variables, but they can also be modeled with other distributions (eg. Gaussian).

GAUSSIAN REWARD



ACTION-VALUES

- Before choosing the strategy, let's keep track of our current estimates of how good pulling lever i is, or in more general terms, how good taking action i is. We calculate

$$Q_t(i) = \frac{\text{Sum of rewards from action i before t}}{\text{Number of times action i is taken prior to t}}$$

for each i and at all times t.

ACTION-VALUES

- This is an estimate of the true action-value $q_*(i)$, which is the expected reward one were to obtain if one were to keep selecting action i.
- By the law of large numbers, we have

$$Q_t(i) \xrightarrow{t \to \infty} q_*(i)$$

for all i.

INCREMENTAL IMPLEMENTATION

$$
\begin{aligned}
Q_{n+1} &= \frac{1}{n+1}\sum_{i=1}^{n+1} R_i \\
&= \frac{1}{n+1}\left(\sum_{i=1}^{n} R_i + R_{n+1}\right) \\
&= \frac{1}{n+1}\left(n\left(\frac{1}{n}\right)\sum_{i=1}^{n} R_i + R_{n+1}\right) \\
&= \frac{1}{n+1}\left(nQ_n + Q_n - Q_n + R_{n+1}\right) \\
&= Q_n + \frac{1}{n+1}\left(R_{n+1} - Q_n\right)
\end{aligned}
$$

INCREMENTAL IMPLEMENTATION

- The formula for incrementally updating the mean estimate is of the form

$$NewEstimate \leftarrow OldEstimate + StepSize[Target - OldEstimate].$$

- The expression $[Target - OldEstimate]$ is called the *error* of the estimate. It is reduced by taking a step in the direction of *Target*.

GREEDY POLICY

- The simplest strategy is the following, at time t, take the action with the best return so far:

$$A_t = \arg\max_i Q_t(i).$$

- This strategy fully exploits, but does not explore.

EXPLOITATION-EXPLORATION

- Exploitation:
  - We exploit our current knowledge of the action-values to select the best action.
  - This is the right thing to do to maximize expected reward in one step, or when there is no more uncertainty.

- Exploration:
  - When we select a non-greedy action, we are exploring.
  - We do so despite receiving a smaller reward in the short term, in the hopes of finding better actions, which we can then exploit at later times to maximize reward in the long run.

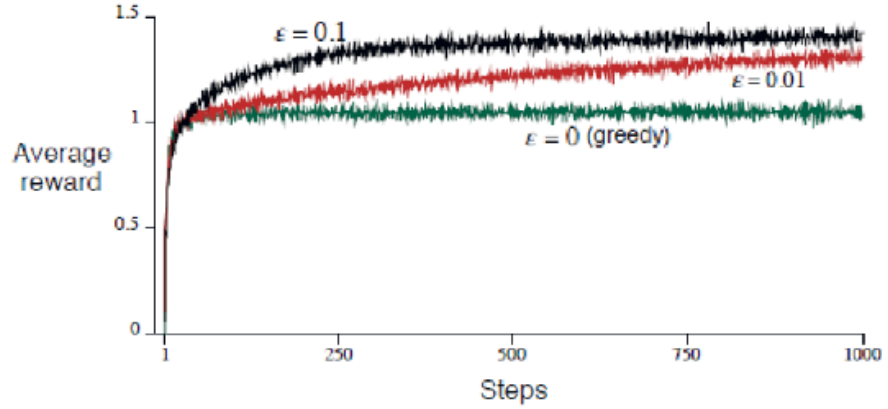
ε-GREEDY POLICY

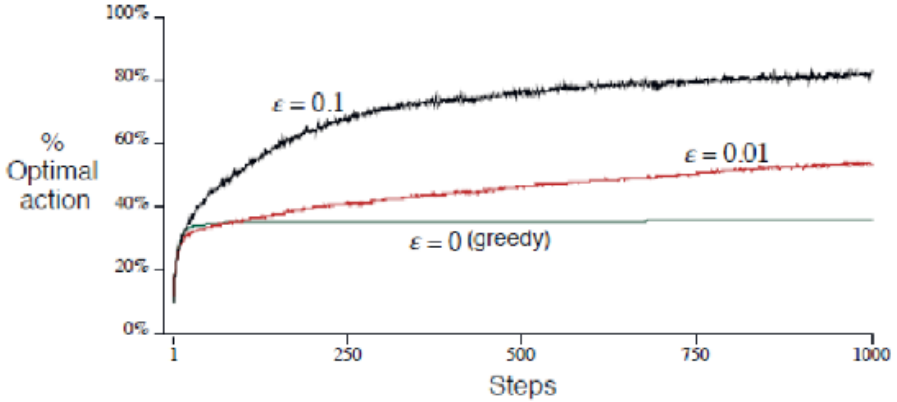
- Behave greedily most of the time, but explore once in a while:

$$A_t = \begin{cases} i^* = \arg\max_i Q_t(i) & \text{with probability } 1 - \epsilon \\ j, j \neq i^* & \text{each with probability } \frac{\epsilon}{k-1} \end{cases}$$

- Balances exploitation vs exploration, but does not select intelligently between the $k - 1$ non-greedy actions.

PERFORMANCE





UPPER CONFIDENCE BOUND

- "Optimism in the face of uncertainty."
- Select action at time t according to

$$A_t = \arg\max_i \left(Q_t(i) + c\sqrt{\frac{\log t}{N_t(i)}} \right).$$

- $N_t(i)$ denotes the number of times action i has been selected prior to time t.
- c is the exploration constant; increasing it favours exploration and decreasing it favours exploitation.

HOEFFDING'S INEQUALITY

Theorem

Let $X_1, \ldots, X_n$ be i.i.d. random variables with mean μ such that $X_i \in [0, 1]$ for all i. Then

$$P\left(\mu \geq \bar{X}_n + U\right) \leq e^{-2nU^2},$$

where $\bar{X}_n = \frac{1}{n}\left(X_1 + \ldots + X_n\right)$.

EXPLAINING EXPLORATION

- We want our upper bound U to be set such that the probability that the true mean exceeds our sample estimate $\bar{X}_n$ plus U is very low.
- We also expect our estimate to get better with time, so, although somewhat arbitrary, we can demand that

$$P\left(\mu \geq \bar{X}_n + U\right) \leq \frac{1}{t^k},$$

for some k. Here t denotes the total number of actions taken so thus far.

EXPLAINING EXPLORATION

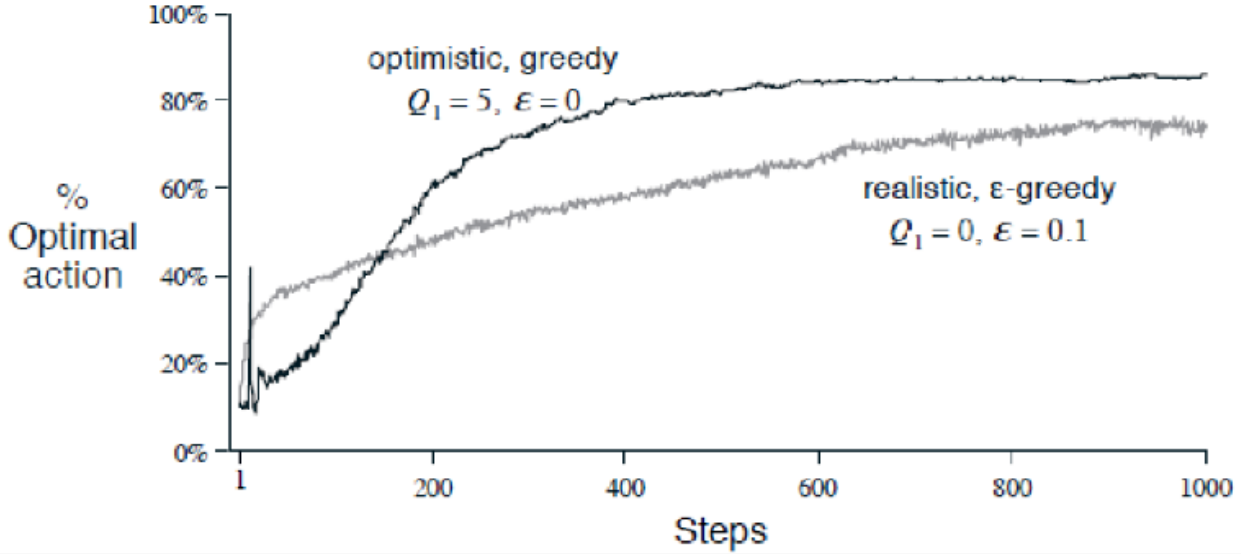
- Thus using Hoeffding's inequality, we require

$$e^{-2nU^2} \leq \frac{1}{t^k} \iff -2nU^2 \leq -k \log t$$

$$\iff U \geq \sqrt{\frac{k}{2}} \sqrt{\frac{\log t}{n}}$$

OPTIMISTIC INITIAL VALUE

- Set $Q_0(i)$ to be something high, rather the expected value of the true means.
- This encourages exploration in the initial stages.



NON-STATIONARY PROBLEMS

- Give more weight to recent rewards than older rewards:

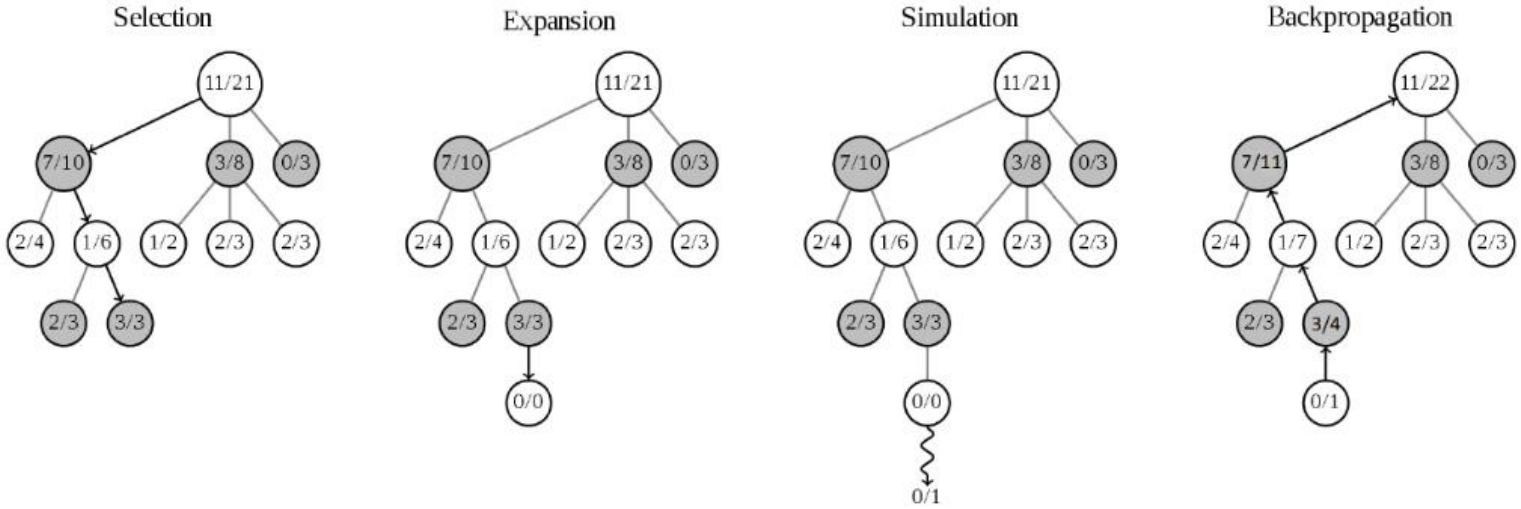
$$Q_{n+1} = Q_n + \alpha[R_{n+1} - Q_n], \qquad \alpha \in (0, 1].$$

- Expanding the expression, we have

$$\begin{aligned}
Q_{n+1} &= Q_n + \alpha[R_{n+1} - Q_n] \\
&= \alpha R_{n+1} + (1-\alpha)Q_n \\
&= \alpha R_{n+1} + (1-\alpha)[\alpha R_n + (1-\alpha)Q_{n-1}] \\
&= \alpha R_{n+1} + (1-\alpha)\alpha R_n + (1-\alpha)^2 Q_{n-1} \\
&= \alpha R_{n+1} + (1-\alpha)\alpha R_n + (1-\alpha)^2 \alpha R_{n-1} \\
&\quad + \cdots + (1-\alpha)^n \alpha R_1 + (1-\alpha)^{n+1} Q_0.
\end{aligned}$$

- This is also called exponential recency-weighted average.

MONTE CARLO TREE SEARCH



- White: Max player, Black: Min player
- Values are recorded from the point of view for the max player white

MONTE CARLO TREE SEARCH

(i) Selection: Starting from the root node, the search process descends down the tree by successively selecting child nodes according to the *tree policy*, the most common of which is UCB1.

(ii) Expansion: When the simulation phase reaches a leaf node, children of the leaf node are added to the tree, and one of them is selected by UCB1.

(iii) Simulation: One random playout, or multiple if parallel processing is employed, is performed until a terminal node is reached.

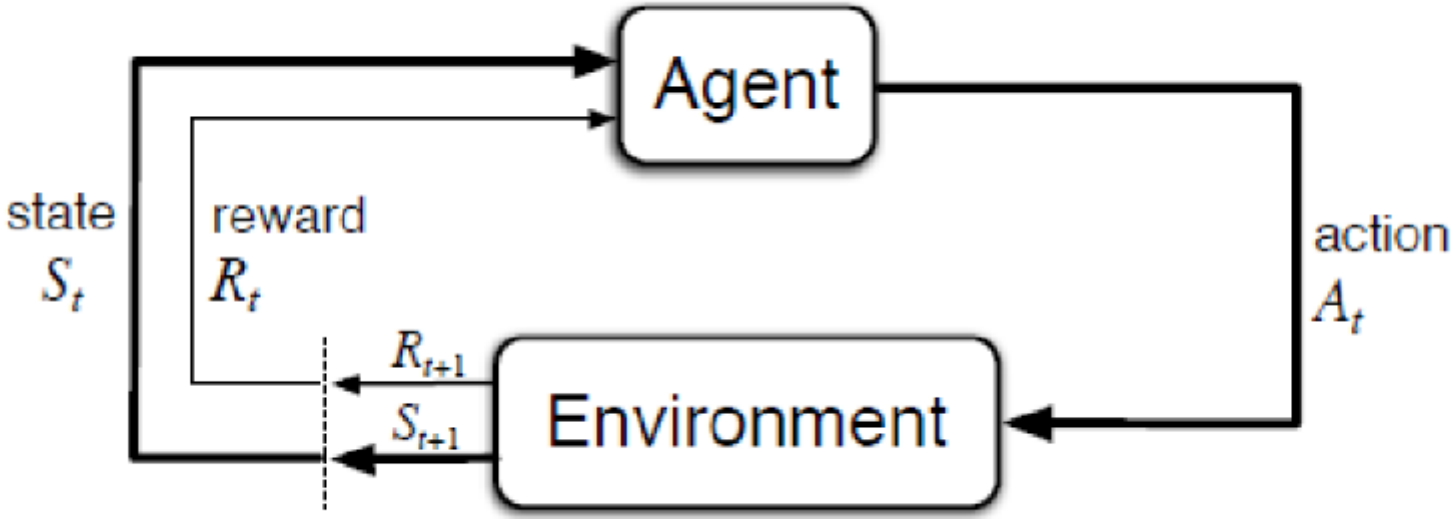
(iv) Back-propagation: The result of the playout is computed and used to update the nodes visited in the selection phase.

MONTE CARLO TREE SEARCH

- Pros:
  - Does not require an evaluation function at leaf nodes.
  - Compared to alpha-beta search, which primarily uses depth-first search, MCTS is a best-first search algorithm, and search can be terminated at any time with relatively good results.
  - More robust than minimax or alpha-beta search.
- Cons:
  - At each node, UCB1 assumes a stationary distribution for the children (actions) of the node, but more often than not the actual distribution is non-stationary.
  - Simple averaging of the rewards to determine the means of the child nodes causes convergence to the optimal action to be slow, particularly in minimax trees.

MARKOV DECISION PROCESS

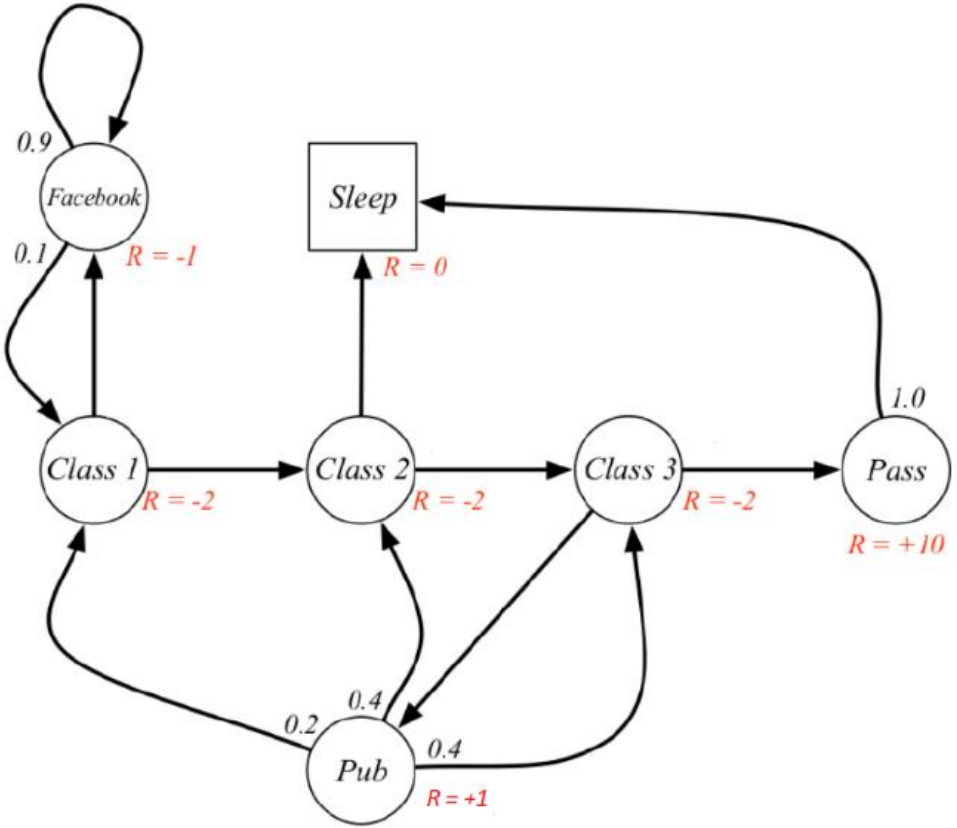
AGENT-ENVIRONMENT



FORMAL DEFINITION

- A Markov decision process is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p)$ where:
  - $\mathcal{S}$ is the set of states
  - $\mathcal{A}$ is the set of actions
  - $\mathcal{R}$ is the set of rewards
  - $p(s', a|s, a) = P(S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a)$ governs the dynamics of the MDP.
- $S_t \in \mathcal{S}$, $R_t \in \mathcal{R}$ and $A_t \in \mathcal{A}$ for all t.
- $\sum_{s' \in \mathcal{S}, r \in \mathcal{R}} p(s', r|s, a) = 1$

STUDENT MDP


0.9
Facebook
0.1
R = -1
Sleep
R = 0
1.0
Class 1
R = -2
Class 2
R = -2
Class 3
R = -2
Pass
R = +10
0.4
0.2
0.4
Pub
R = +1

MARKOVITY

- State-transition probabilities:

$$p(s' \mid s, a) = P(S_t = s' \mid S_{t-1} = s, A_{t-1} = a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a)$$

- Expected rewards:

$$r(s, a) = \mathbb{E}\left[R_t \mid S_{t-1} = s, A_{t-1} = a\right] = \sum_{s' \in \mathcal{S}, r \in \mathcal{R}} p(s', r \mid s, a)$$

OBJECTIVE

- To maximize the expected sum of rewards $E(G_0)$:

$$G_t := R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- $\gamma \in [0, 1]$ is called the discount factor.
- For infinite episodes with bounded rewards, we must have $\gamma < 1$ so that $\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ converges.
- For finite episodes (γ can be 1, no discounting): $G_t = \sum_{k=0}^{T} \gamma^k R_{t+k+1}$.
- $G_t = R_{t+1} + \gamma G_{t+1}$.

POLICY AND VALUE FUNCTIONs

- Policy function $\pi(a|s)$: the probability that $A_t = a$ given that $S_t = s$
- State-value function for policy π:
  - this is the value of a state s under policy π, i.e. the expected return when starting in s and following π thereafter
  - $v_\pi(s) := \mathbb{E}_\pi [G_t \mid S_t = s]$
  - By convention, for terminal states, if any, their value is 0.
- Action-value function for policy π:
  - the value of taking action a in state s under policy π (and thereafter)
  - $q_\pi(s, a) := \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a]$

BELLMAN'S EQUATION FOR POLICY π

$$
\begin{aligned}
v_\pi(s) &\doteq \mathbb{E}_\pi[G_t \mid S_t = s] \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\
&= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r \,|\, s, a) \Big[r + \gamma \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s'] \Big] \\
&= \sum_a \pi(a|s) \sum_{s', r} p(s', r \,|\, s, a) \Big[r + \gamma v_\pi(s') \Big], \quad \text{for all } s \in \mathcal{S},
\end{aligned}
$$

- v_π is the solution to its own Bellman equation
- This is a system of $|\mathcal{S}|$ linear equations in $|\mathcal{S}|$ unknowns.

POLICY EVALUATION

- When $|S|$ is small, we can directly solve the linear equations.
- When the state-space is large, it is more efficient to do iteration using the following update formula:

$$
\begin{aligned}
v_{k+1}(s) &= \mathbb{E}_{\pi}\left[R_{t+1} + \gamma v_k\left(S_{t+1}\right) \mid S_t = s\right] \\
&= \sum_{a} \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma v_k(s')]
\end{aligned}
$$

ITERATIVE POLICY EVALUATION

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π, the policy to be evaluated
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\textit{terminal}) = 0$

Loop:
  $\Delta \leftarrow 0$
  Loop for each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\left[r + \gamma V(s')\right]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

OPTIMAL POLICY FUNCTIONS

- $\pi \geq \pi'$ if and only if $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in \mathcal{S}$.
- Optimal state-value function:

$$v_*(s) := \max_\pi v_\pi(s)$$

for all s.

- Optimal action-value function:

$$q_*(s, a) := \max_\pi q_\pi(s, a)$$

for all s, a.

OPTIMAL BELLMAN EQUATION

$$
\begin{aligned}
v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s,a) \\
&= \max_a \mathbb{E}_{\pi_*}[G_t \mid S_t = s, A_t = a] \\
&= \max_a \mathbb{E}_{\pi_*}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \\
&= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\
&= \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_*(s')\big].
\end{aligned}
$$

OPTIMAL BELLMAN EQUATION

$$
\begin{aligned}
q_*(s,a) &= \mathbb{E}\left[R_{t+1} + \gamma\, v_*(S_{t+1}) \mid S_t = s, A_t = a\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a\right] \\
&= \sum_{s',r} p(s', r|s, a)[r + \gamma \max_{a'} q_*(s', a')]
\end{aligned}
$$

- Both optimal Bellman equations are non-linear and cannot be solved explicitly.

POLICY IMPROVEMENT

- Given the state-value function of a policy π, how do we find a new policy π' that is better than it?
- We can adopt the greedy approach, which does a one-step look-ahead (for each state s):

$$\begin{aligned}\pi'(s) &:= \arg\max_a q_\pi(s,a) \\ &= \arg\max_a \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a\right] \\ &= \arg\max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma\, v_\pi(s')]\end{aligned}$$

POLICY ITERATION

- $\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \cdots \xrightarrow{I} \pi_* \xrightarrow{E} v_*$
- Here, $\xrightarrow{E}$ denotes policy evaluation and $\xrightarrow{I}$ denotes policy improvement.

POLICY ITERATION

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Loop:
   $\Delta \leftarrow 0$
   Loop for each $s \in \mathcal{S}$:
   $v \leftarrow V(s)$
   $V(s) \leftarrow \sum_{s',r} p(s', r|s, \pi(s))\big[r + \gamma V(s')\big]$
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   old-action $\leftarrow \pi(s)$
   $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r|s, a)\big[r + \gamma V(s')\big]$
   If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

VALUE ITERATION

- Combines policy evaluation and improvement into one step.
- Convert optimal Bellman equation into an update rule:

$$v_{k+1}(s) := \max_a \mathbb{E}\left[R_{t+1} + \gamma\, v_k(S_{t+1}) \mid S_t = s, A_t = a\right]$$
$$= \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma\, v_k(s')]$$

VALUE ITERATION

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$

- Greedy policy with respect to optimal value function:

$$\pi(s) \approx \arg\max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma\, v_*(s')] = \arg\max_a q_*(s, a) = \pi_*(s)$$